

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΜΑΚΕΔΟΝΙΑΣ

Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Μάθημα: Τεχνολογία Λογισμικού

Εξαμηνιαία Εργασία — Συνεισφορά σε Έργο Ανοιχτού Κώδικα (ΕΛ/ΛΑΚ)

UoWM Library

Σύγχρονη πλατφόρμα βιβλιοθήκης με βάση το open-source έργο *devrnt/book-library*

Repository ομάδας: github.com/ChrisTypou162734/Library-UoWM

Έργο ΕΛ/ΛΑΚ βάσης: github.com/devrnt/book-library

Ομάδα Έργου

Χρήστος Τύπου [ΑΕΜ]

[Όνομα μέλους 2] [ΑΕΜ]

[Όνομα μέλους 3] [ΑΕΜ]

[Όνομα μέλους 4 — προαιρετικά] [ΑΕΜ]

Ακαδημαϊκό έτος 2025–2026

Πίνακας Περιεχομένων

1. Περιγραφή Έργου

Αντικείμενο της εργασίας είναι η συνεισφορά σε ένα ενεργό έργο ανοιχτού κώδικα και η εφαρμογή ολόκληρου του κύκλου ζωής λογισμικού (απαιτήσεις, σχεδίαση, υλοποίηση, έλεγχος) πάνω σε μια πραγματική βάση κώδικα. Ως έργο βάσης επιλέχθηκε το **devrnt/book-library**, μια εφαρμογή βιβλιοθήκης βιβλίων υλοποιημένη σε Flutter/Dart.

Το αρχικό έργο είναι μια απλή εφαρμογή κινητού (Android/iOS) που επιδεικνύει βασικές λειτουργίες CRUD και το πρότυπο διαχείρισης κατάστασης *Provider / ChangeNotifier*. Τα δεδομένα διατηρούνται τοπικά στη μνήμη, χωρίς backend ή βάση δεδομένων, και η εφαρμογή υποστηρίζει εναλλαγή φωτεινού/σκοτεινού θέματος και προσαρμοστικά widgets για τηλέφωνο και tablet.

Η ομάδα επέκτεινε αυτή τη βάση σε μια **ολοκληρωμένη, πλήρους στοίβας (full-stack) πλατφόρμα βιβλιοθήκης** για τη Βιβλιοθήκη του Πανεπιστημίου Δυτικής Μακεδονίας (UoWM Library). Η συνεισφορά μετασχηματίζει το αρχικό demo σε ένα πραγματικό σύστημα διαχείρισης περιεχομένου (CMS) με τρία υποσυστήματα: δημόσιο δικτυακό τόπο, πίνακα διαχείρισης (admin panel) και REST/WebSocket backend.

1.1 Το έργο βάσης και τα κριτήρια επιλογής

Το book-library πληροί τα κριτήρια επιλογής έργου ΕΛ/ΛΑΚ που ορίζει η εκφώνηση:

Κριτήριο	Τεκμηρίωση για το book-library
≥ 5–10 stars	110 stars
≥ 2 contributors	Πολλαπλοί contributors / 29 forks
Ενεργό repository	55 commits, οργανωμένο ιστορικό
Δέχεται PRs	Ανοιχτό σε forks & pull requests
Έχει documentation	Αναλυτικό README με showcase, οδηγίες, λίστα packages
Όχι φοιτητικό / όχι αλγόριθμος	Πραγματική εφαρμογή Flutter, όχι άσκηση μαθήματος

2. Απαιτήσεις

2.1 Λειτουργικές απαιτήσεις (Functional)

Κωδ.	Περιγραφή
FR-1	Ο επισκέπτης βλέπει το περιεχόμενο της βιβλιοθήκης (αρχική, ανακοινώσεις, συλλογές, υπηρεσίες, πληροφορίες, επικοινωνία).
FR-2	Όλο το περιεχόμενο εμφανίζεται δίγλωσσα (Ελληνικά / Αγγλικά) με εναλλαγή σε πραγματικό χρόνο.
FR-3	Ο χρήστης εναλλάσσει φωτεινό/σκοτεινό θέμα (dark/light mode).
FR-4	Οι ανακοινώσεις εμφανίζονται με σελιδοποίηση (pagination).
FR-5	Ο επισκέπτης υποβάλλει φόρμες (επικοινωνία, αίτηση κάρτας μέλους).
FR-6	Ζωντανή συνομιλία (live chat) μεταξύ επισκέπτη και προσωπικού μέσω WebSocket.
FR-7	Ο διαχειριστής συνδέεται με ασφαλή αυθεντικοποίηση (JWT).
FR-8	Ο διαχειριστής δημιουργεί/τροποποιεί/διαγράφει περιεχόμενο (ανακοινώσεις, παραρτήματα, προσωπικό, υπηρεσίες, συλλογές, σύνδεσμοι, στατιστικά, σελίδες).

FR-9	Ο διαχειριστής μεταφορτώνει εικόνες/αρχεία (media) στο Firebase Storage.
FR-10	Ο διαχειριστής απαντά σε ενεργές συνεδρίες live chat και διαχειρίζεται/κλείνει συνεδρίες.

2.2 Μη λειτουργικές απαιτήσεις (Non-functional)

- **Επεκτασιμότητα:** σαφής διαχωρισμός frontend / backend / αποθήκευσης ώστε κάθε στρώμα να εξελίσσεται ανεξάρτητα.
- **Ασφάλεια:** τα GET endpoints είναι δημόσια· κάθε POST/PUT/PATCH/DELETE προστατεύεται με Bearer JWT και έλεγχο ρόλου admin.
- **Απόδοση:** ασύγχρονη πρόσβαση στη βάση (Motor) και σελιδοποίηση για αποφυγή μεγάλων μεταφορών.
- **Συντηρησιμότητα:** οργάνωση backend σε models / routers / services και τεκμηρίωση API μέσω αυτόματου Swagger UI.
- **Διεθνοποίηση:** πλήρης διγλωσσία σε επίπεδο δεδομένων (μοντέλο BiText), όχι μόνο στο UI.

3. Αρχιτεκτονική

Το σύστημα ακολουθεί αρχιτεκτονική τριών επιπέδων (three-tier): επίπεδο παρουσίασης (δύο Flutter Web εφαρμογές), επίπεδο εφαρμογής (FastAPI) και επίπεδο δεδομένων (MongoDB + Firebase Storage).

3.1 Τεχνολογική στοίβα

Επίπεδο	Τεχνολογίες
Δημόσιος ιστότοπος	Flutter Web (Dart), http, web_socket_channel, font_awesome_flutter, url_launcher
Πίνακας διαχείρισης	Flutter Web (Dart), Provider, shared_preferences (JWT), file_picker, intl
Backend	Python · FastAPI · Uvicorn · Pydantic · python-jose (JWT) · passlib/bcrypt
Δεδομένα	MongoDB (Motor async driver) · Firebase Storage (αρχεία/εικόνες)
Επικοινωνία	REST/JSON · WebSocket (live chat) · CORS

3.2 Δομή του backend

```
uowm_api/   main.py           # FastAPI app + CORS + σύνδεση routers
config.py   # Ρυθμίσεις (.env): MongoDB, Firebase, JWT secret
models/     # Pydantic schemas (BiText, ImageRef, content) routers/
# branches, staff, announcements, services, # content,
forms, chat (WebSocket) services/ # database (Motor), auth (JWT),
firebase scripts/seed.py # Αρχικό γέμισμα της MongoDB
```

Η αυθεντικοποίηση υλοποιείται με JWT (αλγόριθμος HS256, λήξη 8 ωρών). Κάθε προστατευμένο endpoint εξαρτάται από τη συνάρτηση `require_admin`, η οποία επαληθεύει το token και τον ρόλο του χρήστη:

```
@router.post("/") async def create(..., _: str = Depends(require_admin)):
    ...
```

4. Περιγραφή Συστήματος με Διαγράμματα UML

4.1 Διάγραμμα Περιπτώσεων Χρήσης (Use Case)

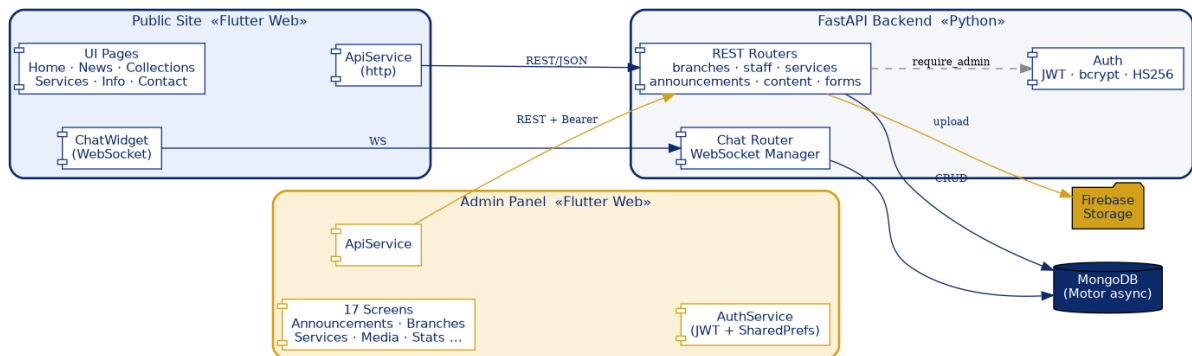
Αποτυπώνει τους δύο ρόλους του συστήματος — τον Επισκέπτη και τον Διαχειριστή — και τις λειτουργίες που εκτελεί ο καθένας.



Εικόνα 1. Διάγραμμα περιπτώσεων χρήσης της πλατφόρμας UoWM Library.

4.2 Διάγραμμα Συστατικών (Component)

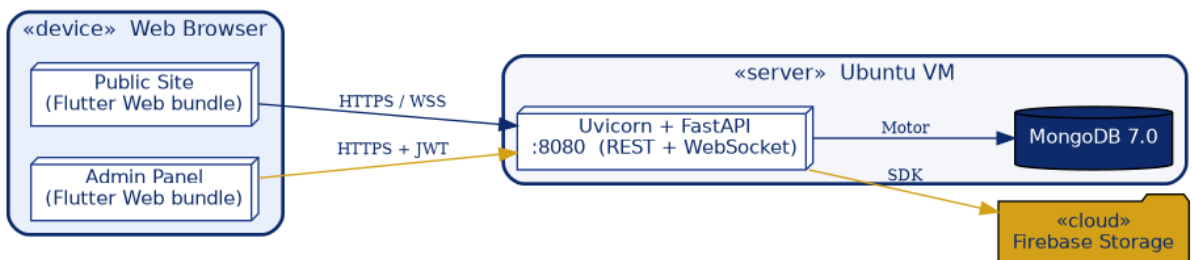
Δείχνει τα συστατικά κάθε επιπέδου και τις διεπαφές μεταξύ τους: τον δημόσιο ιστότοπο, τον πίνακα διαχείρισης, το FastAPI backend, τη βάση MongoDB και την αποθήκευση Firebase.



Εικόνα 2. Διάγραμμα συστατικών — τριεπίπεδη αρχιτεκτονική.

4.3 Διάγραμμα Ανάπτυξης (Deployment)

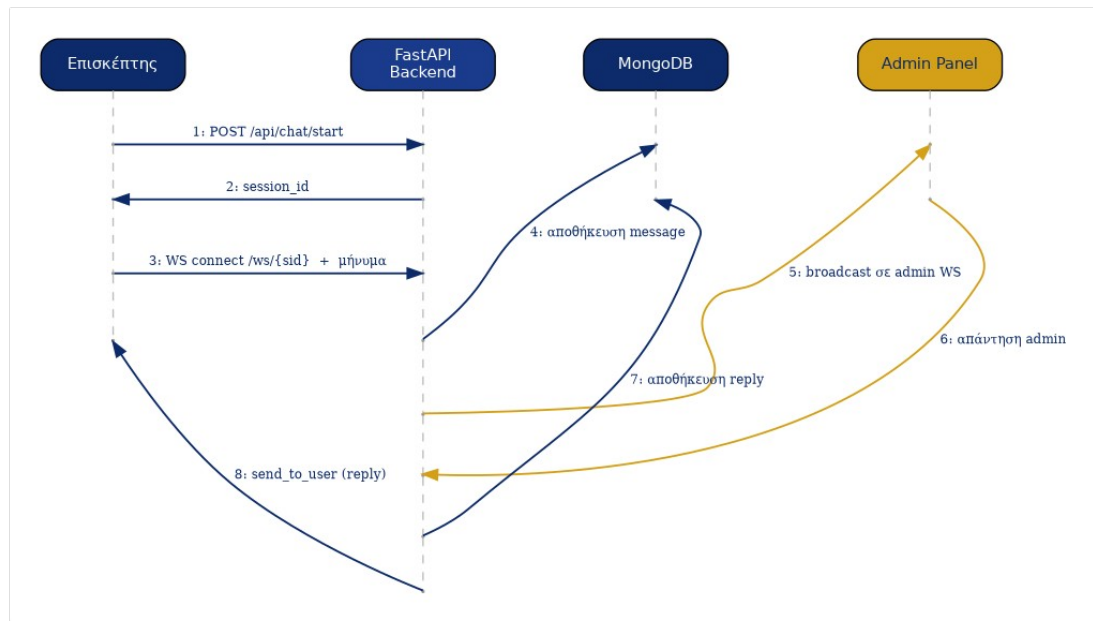
Παρουσιάζει την κατανομή των συστατικών σε κόμβους: τον φυλλομετρητή του χρήστη, τον εξυπηρετητή (Ubuntu VM με Unicorn/FastAPI), τη βάση MongoDB 7.0 και το νέφος Firebase.



Εικόνα 3. Διάγραμμα ανάπτυξης του συστήματος.

4.4 Διάγραμμα Ακολουθίας — Live Chat (WebSocket)

Περιγράφει τη ροή μηνυμάτων της ζωντανής συνομιλίας: ο επισκέπτης ξεκινά συνεδρία, συνδέεται μέσω WebSocket, το backend αποθηκεύει το μήνυμα στη MongoDB και το προωθεί στον διαχειριστή· η απάντηση ακολουθεί την αντίστροφη διαδρομή.



Εικόνα 4. Διάγραμμα ακολουθίας για τη ζωντανή συνομιλία.

5. Επιλογή Συνεισφοράς & Προτοποριακά Στοιχεία

5.1 Η συνεισφορά της ομάδας

Η συνεισφορά εμπίπτει ταυτόχρονα σε όλους τους αποδεκτούς τύπους της εκφώνησης — **νέο feature, βελτίωση λειτουργίας, refactoring και test coverage** — καθώς μετασχηματίζει το αρχικό demo σε πλήρες σύστημα:

- Νέο feature: δίγλωσσο CMS, ζωντανή συνομιλία WebSocket, αυθεντικοποίηση JWT, πίνακας διαχείρισης 17 οθονών, μεταφόρτωση media.
- Βελτίωση λειτουργίας: μετάβαση από τοπική in-memory αποθήκευση σε πραγματική επιμονή δεδομένων με MongoDB και REST API.
- Refactoring: επανασχεδίαση από εφαρμογή ενός επιπέδου (mobile-only) σε καθαρή τριεπίπεδη αρχιτεκτονική με διαχωρισμό ευθυνών.
- Test coverage: widget tests στο frontend και επαλήθευση των endpoints του backend.

5.2 Αιτιολόγηση

Στη φάση σχεδιασμού (Ορόσημο 5) εξετάστηκε η ενσωμάτωση με το Koha OPAC API για άντληση πραγματικών βιβλιογραφικών δεδομένων. Κατά την υλοποίηση, η ομάδα κατέληξε σε ένα δικό της backend (FastAPI + MongoDB): η λύση αυτή δίνει πλήρη έλεγχο στο δίγλωσσο περιεχόμενο, επιτρέπει διαχείριση μέσω admin panel και αποδεσμεύει το έργο από εξωτερική υποδομή Koha. Έτσι η εφαρμογή παραμένει αυτόνομη, ασφαλής και άμεσα επεκτάσιμη — κατάλληλη ως πραγματικός ιστότοπος ακαδημαϊκής βιβλιοθήκης.

5.3 Τι προτοποριακό προσφέρει σε σχέση με το book-library

Ο παρακάτω πίνακας συγκρίνει το αρχικό έργο με την υλοποίηση της ομάδας και αναδεικνύει την προστιθέμενη αξία.

Διάσταση	devrnt/book-library	UoWM Library (ομάδα)
Πλατφόρμα	Mobile app (Android/iOS)	Δημόσιο web + admin web
Αρχιτεκτονική	Ένα επίπεδο, mobile-only	Τριεπίπεδη full-stack
Backend	Κανένα	FastAPI (REST + WebSocket)
Αποθήκευση	In-memory (πητητική)	MongoDB + Firebase Storage
Αυθεντικοποίηση	Καμία	JWT (bcrypt, HS256, ρόλοι)
Διγλωσσία	Όχι	EL/EN σε επίπεδο δεδομένων (BiText)
Real-time	Όχι	Live chat μέσω WebSocket
Διαχείριση περιεχ.	Στατικό demo	CMS με 17 οθόνες admin
Media	Τοπικά assets	Μεταφόρτωση & διαχείριση (Firebase)
Τεκμηρίωση API	—	Αυτόματο OpenAPI / Swagger UI
Έκδοση τεχνολογίας	Flutter 1.7.8 (2019)	Σύγχρονο Flutter / Dart 3 (null-safety)

Συνοπτικά, η ομάδα μετέτρεψε μια επίδειξη του προτύπου Provider σε μια **λειτουργική, ασφαλή και δίγλωσση πλατφόρμα βιβλιοθήκης** με διαχείριση περιεχομένου και επικοινωνία σε πραγματικό χρόνο — στοιχεία που απουσιάζουν εντελώς από το αρχικό έργο.

6. Διαδικασίες Testing

Ο έλεγχος καλύπτει και τα δύο επίπεδα της εφαρμογής:

Frontend (Flutter)

- **Widget tests** (test/widget_test.dart) για επαλήθευση απόδοσης και αλληλεπίδρασης βασικών widgets και των δύο εφαρμογών.
- **Manual / integration έλεγχος** των ροών: εναλλαγή γλώσσας, εναλλαγή θέματος, σελιδοποίηση ανακοινώσεων, υποβολή φόρμας, live chat.

Backend (FastAPI)

- **Έλεγχος endpoints** μέσω του ενσωματωμένου Swagger UI (/docs) — δημόσια GET και προστατευμένα POST/PUT/DELETE.
- **Έλεγχος αυθεντικοποίησης**: επιβεβαίωση ότι τα προστατευμένα endpoints απορρίπτουν αιτήματα χωρίς έγκυρο token (401/403).
- **Bug reproduction**: καταγραφή και αναπαραγωγή σφαλμάτων (π.χ. CORS, αναντιστοιχία base URL) πριν τη διόρθωση.

Εντολές εκτέλεσης ελέγχων:

```
flutter test          # widget tests (frontend)
reload               # τοπικό backend + Swagger στο /docs
unicorn main:app --
```

7. Δυσκολίες

Διγλωσσία σε επίπεδο δεδομένων

Η υποστήριξη EL/EN δεν περιορίστηκε σε μεταφράσεις UI· απαιτήσε σχεδιασμό του μοντέλου BiText {el, en} σε κάθε πεδίο περιεχομένου και συνεπή χειρισμό σε frontend και backend.

WebSocket live chat

Η διαχείριση πολλαπλών ταυτόχρονων συνεδριών (χρήστες + διαχειριστές) μέσω ενός in-memory ConnectionManager, με ταυτόχρονη αποθήκευση ιστορικού στη MongoDB, ήταν το πιο σύνθετο κομμάτι.

Αυθεντικοποίηση & ασφάλεια

Σωστή ρύθμιση JWT (έκδοση/επαλήθευση token), hashing με bcrypt και προστασία μόνο των endpoints εγγραφής χωρίς να επηρεάζεται η δημόσια ανάγνωση.

CORS & ρυθμίσεις δικτύου

Η επικοινωνία Flutter Web ↔ FastAPI απαιτήσε σωστό CORS και ευθυγράμμιση του base URL μεταξύ δημόσιου site, admin panel και backend.

Μετάβαση από τον αρχικό σχεδιασμό

Η εγκατάλειψη της ενσωμάτωσης Koha υπέρ ενός custom backend απαιτήσε επανασχεδίαση των ροών δεδομένων, αλλά οδήγησε σε πιο αυτόνομο και ελεγχόμενο σύστημα.

8. Οργάνωση GitHub & Συμπεράσματα

8.1 Πρακτικές GitHub

- Fork του έργου βάσης και ξεχωριστά feature branches ανά λειτουργία.

- Issues για παρακολούθηση εργασιών και bugs, σύνδεση με τα ορόσημα (milestones) της εκφώνησης.
- Περιγραφικά commits και τεκμηριωμένα pull requests με code review μεταξύ των μελών.
- README ανά υποσύστημα (libraryuowm, libraryuowmadmin, uowm_api) ως ζωντανή τεκμηρίωση.

8.2 Συμπεράσματα (Lessons learned)

Η εργασία έδειξε έμπρακτα πώς ένα μικρό open-source demo μπορεί να αποτελέσει αφετηρία για ένα πραγματικό, πολυεπίπεδο σύστημα. Η ομάδα εφάρμοσε όλο τον κύκλο ζωής λογισμικού — από την ανάλυση απαιτήσεων και τη μοντελοποίηση UML έως την υλοποίηση, τον έλεγχο και τη συνεργατική ανάπτυξη μέσω GitHub — αποκτώντας εμπειρία σε αρχιτεκτονική full-stack, ασφάλεια, διεθνοποίηση και επικοινωνία σε πραγματικό χρόνο.

Σύνδεσμοι: [Repository ομάδας](#) · [Έργο βάσης](#)